



Sujet : Travail Pratique 2 : Application de nouvelles
Valeur : 30% du semestre
Date de remise : 7 décembre 2026
Mode de remise : remise automatique sur GitHub

Instructions générale

- ❖ L'API 28 (Pie, Android 9.0) doit être le SDK minimum
- ❖ Concevoir une interface **adaptative** avec **Jetpack Compose** (portrait/paysage).
- ❖ Implémenter la **navigation Compose** entre plusieurs écrans.
- ❖ Gérer la **persistance locale** (Room).
- ❖ Intégrer **Firestore** pour la persistance distante (profil utilisateur et favoris).
- ❖ Consommer une **API REST publique** (<https://newsdatahub.com/>) pour récupérer des articles de nouvelles.
- ❖ Appliquer un **thème visuel personnalisé** conforme aux principes de Material Design 3.
- ❖ Structurer une application complète autour d'un **Scaffold** avec :
 - **TopBar** pour le titre ou les actions principales
 - **BottomBar** pour la navigation
 - **FloatingActionButton (FAB)** pour une action clé (ex. : rafraîchir, etc.)
- ❖ Le travail se réalise **en équipe de deux (2)**.

Contexte

L'objectif de ce travail pratique sera de programmer une application de nouvelles. Vous pouvez vous inspirer d'applications déjà existantes.

Voici quelques applications pour vous inspirer :

- ❖ La Presse+
- ❖ Ground News
- ❖ Google News
- ❖ Apple News

Liste d'articles

La première étape du projet serait de faire une liste qui utilise une *LazyList*. Chaque item de la liste devrait présenter le titre, la source et la date de l'article.

- ❖ *LazyList* avec une liste des articles
- ❖ Les éléments doivent être récupérés de l'API (<https://newsdatahub.com/>)
- ❖ Chaque item contient le **titre**, la **source** et la **date** de l'article

- ❖ Cliquer sur un item de la liste devrait me permettre de le **voir en détail**

Détail d'un article

Lorsque nous cliquons sur un article dans la liste, nous devrions être redirigés vers une page qui permet de voir tous ses détails.

- ❖ Cliquer sur un article de la liste devrait me permettre de le **voir en détail**
- ❖ Devrait présenter le **titre, image, description, auteur, pays, source, url de l'article original**

Rafraîchir les articles

Un Floating Action Button (FAB) devrait permettre de rafraîchir les articles présentés. Room doit contenir et afficher les nouvelles dernièrement affichées.

- ❖ Un Floating Action Button permet de rafraîchir les nouvelles
- ❖ Ces nouvelles devraient être sauvées dans Room
- ❖ Tant que je ne rafraîchis pas, les données de Room sont utilisées

Configuration de l'application

Une nouvelle page, accessible à partir d'une icône dans le Top ou BottomBar, devrait permettre à l'utilisateur de configurer le type de nouvelle qu'il veut voir.

- ❖ Sélectionner **les catégories** d'articles
- ❖ Sélectionner **les pays** sources des articles (ex. : CA, US, FR, etc.)
- ❖ Sélectionner **la langue** des articles
- ❖ Ces préférences devraient être sauvegardés sur Firebase.

L'application devra permettre :

1. **La connexion de l'utilisateur** (Firebase Authentication).
2. **L'application d'un thème personnalisé** avec une palette de couleurs, typographie et formes définies par l'équipe.

Intégration de l'API NewsDataHub

1. Création d'une clé d'API personnelle

Chaque équipe doit :

- Se créer un compte gratuit sur <https://newsdatahub.com>.
- Générer **sa propre clé d'API**.
- Configurer cette clé dans un **fichier sécurisé** (ex. : local.properties ou via BuildConfig).

2. Gestion du quota et bonnes pratiques

Le plan gratuit impose une **limite de requêtes journalières**.

Chaque équipe doit :

- Respecter ce quota (ne pas déclencher d'appels API en boucle).
- Implémenter un **mécanisme de mise en cache locale** (Room) pour éviter les appels redondants.
- Rafraîchir les données uniquement :
 - lors d'une action utilisateur (ex. : bouton "Rafraîchir")
 - ou après un délai raisonnable (ex. : 1 heure).
- Gérer les **erreurs réseau** et les **réponses vides** de manière élégante (message ou Snackbar).

3. Paramètres de la requête

Votre application doit construire dynamiquement la requête selon les préférences de l'utilisateur :

- **Catégories** sélectionnées
- **Pays** choisis
- **Langue** de nouvelles à inclure

Affichage des détails d'un article

Lorsque nous cliquons sur un article dans la liste, nous devrions être redirigés vers une page qui affiche les détails de l'article :

- ❖ Le titre
- ❖ L'image
- ❖ La date
- ❖ La catégorie
- ❖ La description
- ❖ Le contenu
- ❖ Le lien vers l'article complet (à ouvrir dans le navigateur)

Utilisation de la librairie *Room*

La prochaine étape du projet serait d'instaurer une librairie qui permettrait de sauver l'information localement pour ne pas perdre ses modifications. La librairie requise est *Room*. Les nouvelles chargées de l'API sont enregistrées automatiquement dans Room.

- ❖ Capable de faire des modifications qui sont **sauvegardées localement**
- ❖ Ne pas perdre les modifications lors du changement de **portrait à paysage**

Autres requis techniques

Voici une courte liste de requis technique que vous devez respecter :

- ❖ Séparer l'écran avec un **Scaffold**, il vous faut au moins 1 *TopBar* et/ou 1 *BottomBar*
- ❖ Confirmer les actions de l'utilisateur avec des **Snackbar** (Édition, suppression, etc.) avec possibilité d'annuler les modifications.
- ❖ L'application doit utiliser les ressources pour **fonctionner en anglais et en français**

Fonctionnalités supplémentaires

Les pages précédentes contiennent les fonctionnalités les plus importantes, cependant, il manque beaucoup de choses pour faire une application complète. Vous allez devoir décider d'ajouter les fonctionnalités qui, selon vous, manquent à l'application. Voici une liste d'idée non exhaustive :

- ❖ Ajouter des items à une liste de favoris
- ❖ Filtrer la liste selon leurs attributs, ascendants et descendants
- ❖ Ajouter du son et des animations

Barème

Critères	Points
Fonctionnement du programme - Récupérer et afficher les éléments à partir de l'API - Changer les images avec coil - Sauvegarder l'information localement avec Room - Sauvegarder dans Firebase	/50
Fonctionnalités supplémentaires - Ajout de fonctionnalités supplémentaires - Fonctionnement et homogénéité des fonctionnalités	/15
Ergonomie / qualité de l'interface - Application utilisable en portrait et paysage - Facilité d'utilisation et aspect visuel - Respect des normes d'ergonomie d'Android - Utilisation correct des Snackbars	/15
Programmation - Clarté du code (ajout de commentaire ou nécessaire) - Respect des normes de programmation présentées - Nom de classe, fonctions et variables claires - Éviter la duplication de code et le franglais - Utilisation des ressources (en / fr) - Séparation de code en composables réutilisables	/15
ReadMe - Un ReadMe avec une description des tâches de chaque coéquipier - Une section sur les fonctionnalités requises manquantes - Une section sur les fonctionnalités supplémentaires	/5
Total	/100